

This code implements a sophisticated **Agent-Based Model (ABM)** of a stock market. Instead of just modeling prices as a single line, it simulates the individual behaviors of different types of traders (agents) and how their collective actions, combined with random "noise," drive the market.

Here is a detailed breakdown of the components.

## 1. Core Data Structures and Enums

The code begins by defining the "rules of the world" using Enums and data containers. This ensures type safety and a clean way to represent the states of the market.

```
class OrderType(Enum):
```

```
    BUY = "BUY"
```

```
    SELL = "SELL"
```

```
    HOLD = "HOLD"
```

```
class AgentType(Enum):
```

```
    RANDOM = "RANDOM"
```

```
    MEAN_REVERSION = "MEAN_REVERSION"
```

```
    TREND_FOLLOWER = "TREND_FOLLOWER"
```

```
    NOISE = "NOISE"
```

- **Enums:** These categorize orders, agents, and statuses, making the code readable (e.g., using `OrderType.BUY` instead of just a string or number).
- **MarketData:** This acts as a central repository that tracks price histories, calculates Moving Averages, and monitors volatility. It serves as the "information feed" that agents look at before making a decision.

## 2. The Asset Model (The Stock Class)

The Stock class simulates the price movement of an individual company. It uses a mathematical model called **Geometric Brownian Motion (GBM)** with a mean-reversion twist.

```
def update_price(self, market_sentiment: float = 0.0):
```

```
    drift = 0.0001
```

```
    random_shock = np.random.normal(0, self.volatility)
```

```
    # P_t = P_{t-1} * exp(drift + random_shock + sentiment)
```

```
    price_change = drift + random_shock + (market_sentiment * 0.001)
```

```
    self.current_price *= np.exp(price_change)
```

```
    # Add some mean reversion
```

```
reversion_strength = 0.01
self.current_price = self.current_price * (1 - reversion_strength) + self.base_price *
reversion_strength
```

- **Drift:** A small constant that represents the long-term expected growth of a stock.
- **Random Shock:** Simulates the daily "noise" or unpredictability of the market.
- **Market Sentiment:** If the overall market is doing well, this pushes the price up.
- **Mean Reversion:** A "rubber band" effect that pulls the price back toward its base\_price if it strays too far, simulating the idea that stocks have an intrinsic value.

## 3. Trading Strategies (The "Brains")

The simulator uses the **Strategy Pattern**. It defines an abstract base class (TradingStrategy) and then implements three specific logic types:

### A. Mean Reversion

This strategy assumes that if a price is too far from its average, it will eventually return to that average.

```
z_score = (current_price - avg_price) / std_price if std_price > 0 else 0
```

```
if z_score < -self.threshold:
    return TradingSignal(OrderType.BUY, confidence)
elif z_score > self.threshold:
    return TradingSignal(OrderType.SELL, confidence)
```

- It calculates a **Z-score** (how many standard deviations the price is from the mean).
- If the price is significantly "cheap" (negative Z-score), it buys; if "expensive" (positive Z-score), it sells.

### B. Trend Following

This strategy follows the momentum. It uses two Moving Averages (MA): a short-term and a long-term one.

```
if short_ma > long_ma * 1.01: # Uptrend
    return TradingSignal(OrderType.BUY, confidence)
elif short_ma < long_ma * 0.99: # Downtrend
    return TradingSignal(OrderType.SELL, confidence)
```

- If the short-term MA crosses above the long-term MA (a "Golden Cross" logic), it signals a **BUY**, betting that the upward trend will continue.

## 4. The StockAgent Class

This represents the individual investor. Every agent has their own cash balance, a portfolio of stocks they own, and a strategy.

```
def update_portfolio(self, stock_symbol: str, quantity: int, price: float, is_buy: bool):
    if is_buy:
        total_cost = quantity * price
        if self.cash >= total_cost:
            self.cash -= total_cost
            self.portfolio[stock_symbol] = self.portfolio.get(stock_symbol, 0) + quantity
    else:
        # Sell logic...
```

- **Portfolio Management:** The agent tracks their wealth\_history over time, which is used at the end of the simulation to calculate performance metrics like the **Sharpe Ratio** (risk-adjusted return).

## 5. The Orchestrator (StockMarketSimulator)

This class manages the interaction between the agents and the stocks. It runs the "main loop" of the market.

```
def run_step(self):
    self.current_step += 1
    # 1. Update stock prices
    # 2. Update market sentiment based on recent trends
    # 3. Agents generate orders
    # 4. Execute orders (matching agents with market prices)
    # 5. Record trading volume
```

- **Market Sentiment:** This is an aggregate score. If most stocks rose in the last 5 steps, the sentiment becomes positive, which in turn influences the next price update, creating a feedback loop.
- **Order Execution:** Orders aren't just executed at the perfect price; the simulator adds "slippage" (random noise in the execution price) to mimic real-world market friction.

## 6. Visualization and Reporting

The final part of the code uses matplotlib to create a dashboard of the market's health.

- **Stock Price Evolution:** A line chart of all stocks.
- **Agent Wealth:** Shows which strategies are winning.

- **Portfolio Composition:** A stacked bar chart showing what each agent currently owns (Cash vs. Stocks).
- **Data Export:** It saves the results to stock\_prices.csv and agent\_wealth.csv using pandas so you can analyze the data in Excel or another tool later.